



ΑΡΙΣΤΟΤΕΛΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΟΝΙΚΗΣ

Πολυτεχνική Σχολή

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών

ΜΙΚΡΟΕΠΕΞΕΡΓΑΣΤΕΣ ΚΑΙ ΠΕΡΙΦΕΡΕΙΑΚΑ

Εργαστήριο 2 - Μάιος 2026

ΟΜΑΔΑ 33

-

Λουλουδάκη Αγγελική

AEM : 10976

Γούναρης Χρήστος

AEM : 10638

1 Υλοποίηση και Λογική Λειτουργίας

1.1 Χρονισμός και Καταστάσεις Συστήματος

Αρχικά ορίζουμε έναν `timer` που παράγει διακοπές κάθε 1ms. Σε κάθε μία διακοπή αυξάνει την καθολική μεταβλητή `sys_ticks` κατά ένα. Η μεταβλητή αυτή χρησιμοποιείται ως το κεντρικό ρολόι του συστήματος, καθώς συγκρίνεται με αποθηκευμένες τιμές (time stamps) για τον υπολογισμό χρονικών διαστημάτων.

Για τον έλεγχο της ροής του προγράμματος, ορίζουμε τη δομή `SystemState` με τις παρακάτω καταστάσεις, επιτρέποντας τον ασύγχρονο έλεγχο της εφαρμογής:

1. `STATE_IDLE`: Αναμονή για είσοδο εντολής.
2. `STATE_EXECUTING_PROFILE`: Ενεργή εκτέλεση προφίλ αναλαμπών.
3. `STATE_ESTOP`: Κατάσταση έκτακτης ανάγκης (Emergency Stop).
4. `STATE_ESTOP_AWAITING_PWD`: Αναμονή κωδικού ξεκλειδώματος μετά το E-Stop.

Στη συνέχεια, αρχικοποιούνται τα περιφερειακά και ρυθμίζονται οι προτεραιότητες των διακοπών (NVIC) ως εξής:

- **Προτεραιότητα 1:** Κουμπί E-Stop (EXTI).
- **Προτεραιότητα 2:** Timer (System Tick).
- **Προτεραιότητα 3:** UART (Επικοινωνία).

1.2 Διαχείριση Εισόδου και UART

Το σύστημα λαμβάνει εντολές μέσω της διακοπής `UART RX Interrupt`. Κάθε χαρακτήρας αποθηκεύεται προσωρινά σε έναν ενδιάμεσο `buffer (rx_buffer)`. Κατά τη λήψη:

- Θέτουμε τη σημαία `rx_active = 1`, υποδηλώνοντας ότι μια εισαγωγή βρίσκεται σε εξέλιξη.
- Η μεταβλητή `last_rx_time` παίρνει την τρέχουσα τιμή των `sys_ticks`, κάνοντας `reset` στον μετρητή αδράνειας των 4 δευτερολέπτων.

Αν ο χαρακτήρας που εισαχθεί είναι ο `Enter`, η εντολή θεωρείται πλήρης. Προστίθεται ο τερματικός χαρακτήρας `\0` στον `rx_buffer`, η σημαία `rx_complete` γίνεται 1 και η `rx_active` μηδενίζεται. Σε περίπτωση που ληφθεί νέα εντολή κατά την εκτέλεση ενός ήδη υπάρχοντος προφίλ, το τρέχον προφίλ διακόπτεται και αντικαθίσταται αμέσως.

1.3 Εκτέλεση Προφίλ και Έλεγχος LED

Στην κύρια λούπα (`main loop`), όταν εντοπιστεί `rx_complete == 1`, καλείται η συνάρτηση `start_profile_execution()`. Αυτή η συνάρτηση:

1. Αντιγράφει τον `buffer` σε έναν μόνιμο πίνακα και υπολογίζει το μήκος του.
2. Ελέγχει αν το τελευταίο σύμβολο είναι παύλα (`' '`). Αν είναι, θέτει `loop_profile = 1` (επανάληψη), αλλιώς `loop_profile = 0`.
3. Μηδενίζει τον δείκτη ψηφίων `profile_digit_index` και θέτει την κατάσταση σε `STATE_EXECUTING_PROFILE`.
4. Συγχρονίζει το `profile.start_time` με τα `sys_ticks`.

Παράλληλα, εξασφαλίζεται η προστασία από αδράνεια: αν $sys_ticks - last_rx_time \geq 4000ms$, το σύστημα αντιδρά αναλόγως.

Όταν το σύστημα βρίσκεται στην κατάσταση εκτέλεσης, η συνάρτηση `process_blinking()` ελέγχει τον χαρακτήρα `current_profile[profile_digit_index]`.

- Αν ο χαρακτήρας είναι `'0'`, το LED σβήνει.
- Σε κάθε άλλη περίπτωση, υπολογίζεται το `toggle_interval`. Μόλις η διαφορά $sys_ticks - led_toggle_time$ υπερβεί το διάστημα αυτό, το LED αλλάζει κατάσταση (`toggle`).

Κάθε 2 δευτερόλεπτα, ο δείκτης `profile_digit_index` αυξάνεται. Μόλις ολοκληρωθεί το προφίλ, αν `loop_profile == 1`, η εκτέλεση ξεκινά από την αρχή, αλλιώς το σύστημα επιστρέφει σε `STATE_IDLE`.

1.4 Λειτουργία Έκτακτης Ανάγκης (E-Stop)

Με το πάτημα του κουμπιού (EXTI), η κατάσταση μεταβαίνει αμέσως σε `STATE_STOP` και το LED ανάβει σταθερά. Ο δείκτης `rx_index` μηδενίζεται για να καθαρίσουν ημιτελείς εντολές.

Με δεύτερο πάτημα του κουμπιού, μεταβαίνουμε στην κατάσταση `STATE_STOP_AWAITING_PWD`, αποθηκεύοντας τη χρονική στιγμή στο `estop_override_time`. Το σύστημα περιμένει τον κωδικό "UNLOCK" για 5000ms. Αν ο χρόνος παρέλθει χωρίς επιτυχή είσοδο, το σύστημα επιστρέφει στο `STATE_STOP`. Σε περίπτωση έκτακτης και σωστής εισαγωγής, το σύστημα επανέρχεται στην κατάσταση `STATE_IDLE`.

2 Δοκιμές και Έλεγχος (Testing)

Για την επαλήθευση της ορθής λειτουργίας του συστήματος, πραγματοποιήθηκαν εκτεταμένες δοκιμές σε όλα τα υποσυστήματα. Αρχικά, ελέγχθηκε η λήψη των Προφίλ Λειτουργίας μέσω της UART, εισάγοντας ακολουθίες όπως "12-" και "4152". Η ακρίβεια των συχνοτήτων (1-9 Hz) και η εναλλαγή των ψηφίων ανά ακριβώς 2 δευτερόλεπτα επαληθεύτηκαν μέσω οπτικού ελέγχου στο LED και ανάλυσης της κυματομορφής εξόδου. Επιπλέον, πραγματοποιήθηκε έλεγχος των μηχανισμών ασφαλείας, επιβεβαιώνοντας ότι η εισαγωγή χαρακτήρων ακυρώνεται μετά από 4 δευτερόλεπτα αδράνειας, καθώς και ότι η κατάσταση Emergency Stop (E-Stop) ενεργοποιείται ακαριαία με το πάτημα του κουμπιού, κλειδώνοντας το UART και κρατώντας το LED σταθερά αναμμένο. Τέλος, δοκιμάστηκε η διαδικασία ξεκλειδώματος, όπου το σύστημα επέστρεφε σε αδράνεια μόνο μετά το δεύτερο πάτημα του κουμπιού και την αποστολή της λέξης "UNLOCK" εντός του χρονικού ορίου των 5 δευτερολέπτων. Η επικοινωνία μεταξύ υπολογιστή και πλακέτας έγινε με την χρήση του προτινόμενου προγράμματος Tera Term.

3 Προβλήματα και Αντιμετώπιση

Κατά τη διάρκεια της ανάπτυξης του firmware, το κυριότερο πρόβλημα που αντιμετωπίσαμε αφορούσε τη λειτουργία του διακόπτη E-Stop σε συνδυασμό με τον παρεχόμενο driver `gpio.c`. Παρατηρήθηκε ότι η διακοπή EXTI δεν εκτελούσε το callback κατά το πάτημα του μπλε κουμπιού της πλακέτας Nucleo. Μετά από ανάλυση του κώδικα του driver, εντοπίστηκε ότι η συνάρτηση `EXTI15_10_IRQHandler` έλεγχε αν η κατάσταση του pin ήταν HIGH (`IDR & bit`) για να καλέσει το callback, ενώ το φυσικό κουμπί είναι Active-Low. Προκειμένου να επιλυθεί το ζήτημα χωρίς να τροποποιηθούν οι drivers, ρυθμίσαμε το trigger στη `main.c` ως `Rising`, ώστε η διακοπή να αναγνωρίζεται κατά την απελευθέρωση του κουμπιού. Ένα δευτερεύον ζήτημα αφορούσε πολυάριθμα warnings κατά τη μεταγλώττιση (π.χ. *non-portable path*), τα οποία οφείλονταν σε διαφορές πεζών-κεφαλαίων στα ονόματα των αρχείων επικεφαλίδας (headers) της βιβλιοθήκης Standard Peripheral Library, ωστόσο δεν επηρέαζαν την εκτέλεση του κώδικα.